PROJECT-BASED LEARNING

PHASE-II EVALUATION

Disaster Management System

Team ID: DSCPP-III-2026-T058

Department of Computer Science & Engineering
Graphic Era (Deemed to be University), Dehradun
Academic Session 2026–27

Team & Mentor Details

Team Information

Team ID: DSCPP-III-2026-T058
Semester: 3rd
Section: B, E, F, G
Domain: *Data Structures and C++*

Team Members

1. *Avika Mittal – 2510012768*
2. *Gaurav Singh Negi – 2510012516*
3. *Avni Khandelwal – 25103140050*
4. *Aryan Gautam – 2510012654*

Mentor

Mr. Navneet Rajpoot

Interactions completed: 6

Phase-I Recap & Feedback Incorporated

Phase-I Status

The project was suggested for organizing information related to disasters including people who are affected, rescue teams, available resources, emergencies, etc. The project goals, architectural design, technology used, and roles were defined.

Feedback Received

- *Use DSA concepts to implement practical features rather than only storing and managing data.*
- *Incorporate algorithms to improve the efficiency of emergency request handling and rescue route planning.*

Changes Incorporated

- *Separate C++ classes and modules for different system operations.*
- *Emergency request handling using priority and shortest path navigation using graphs.*
- *Added resource allocation, action-history tracking and file-based data storage.*

Problem Statement & Finalized Objectives

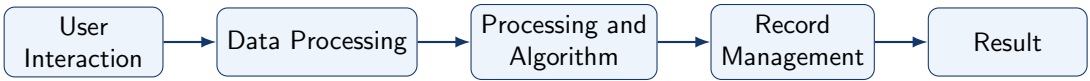
Problem Statement

Managing information about affected people, rescue teams, resources, emergency requests and locations can become difficult and unorganized. The project addresses this problem through a centralized C++ system that supports storing, updating, searching and managing disaster-related information.

Finalized Objectives

1. *Develop a functional system for managing disaster-related information.*
2. *Manage affected people, disasters, rescue teams and resources.*
3. *Apply DSA concepts, including stack, sorting and graph-based shortest-path calculation.*
4. *Integrate OOPs concepts through modular classes, inheritance and polymorphism.*

Updated System Architecture / Workflow



Major Components

- *Disaster and affected-person management*
- *Rescue team and resource management*
- *Emergency request prioritization*
- *Route navigation and shortest-path calculation*
- *Action history and file handling*

Changes Since Phase-I

- *Expanded the basic disaster module into a multi-module C++ application.*
- *Introduced a bubble sorting to process emergency requests by urgency.*
- *Added resource inventory management and action-history tracking.*

Module Design & Functional Requirements

Major Modules

1. *Disaster Management Module*
2. *Resource Management Module*
3. *Emergency Request Module*
4. *Route Navigation Module*
5. *History Module*

Functional Requirements

- *Register, update, and view disaster records, manage affected people and rescue teams.*
- *Add resources, allocate quantities, view inventory.*
- *Register, prioritize, process emergency requests.*
- *Add routes and calculate shortest paths.*
- *Record and display recent system actions.*

Module Dependency

The Disaster Management System acts as the central controller. It contains the modules and coordinates user operations. The history module records selected actions, while the file-handling function saves affected-person records.

Technology Stack & Development Methodology

Technology Stack

- Programming: *C++*
- Database: *File-based Storage (.txt)*
- Framework: *C++ STL, fstream*
- Development Tool: *VS Code/ Code::Blocks*
- Version Control: *Git / GitHub*

Development Methodology

1. Identify requirements
2. Design system structure
3. Develop using C++ and DSA
4. Test system functions
5. Evaluate results

C++ is selected for easy implementation of DSA and OOP concepts, while the step-by-step methodology ensures systematic development, testing, and evaluation of the system.

Implementation of PBL Course Concepts

Course	Concepts Actually Implemented	Project Module
Data Structures in C	*Vector, Linked List, Bubble sort, Graph, Dijkstra's Algorithm, Stack*	*Route,Emergency,Disaster Module*
OOPs with C++	*Classes and Objects, Encapsulation, Abstraction*	*Audit,Resource Module*

Development & Implementation Progress

Module	Status	Work Completed
Disaster Management	Completed	*Register, update, view, manage affected people and rescue teams*
Resource Management	Completed	*Add, allocate, view stock*
Emergency Requests	Completed	*Register requests and Processes the highest-priority request first*
Route Navigation	Pending	*Add routes, find direct routes, and calculate the shortest route and distance*
Audit History	Completed	*Record important actions and display the latest action first.*

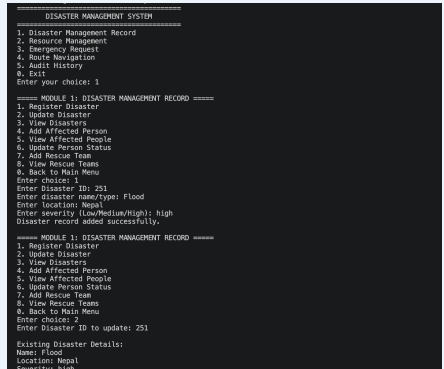
Overall Project Completion: *80%*

Working Prototype / Demonstration

Implemented Features

- *Register, update and view disaster records; manage affected people, their status and rescue teams.*
- *Add, update and view available resources*
- *Arrange requests by priority and process the highest-priority request first.*
- *Add routes, check direct connections and calculate the shortest route.*
- *Record system actions and display the latest actions first. Save system data to a text file.*

Prototype / Output



Testing, Results & Validation

Test / Module	Expected Result	Observed Result	Status
Disaster Management	*Register, update and display disaster records*	*Records are added, updated and displayed*	Pass
Resource Management	*Add and allocate resources*	*Stock updated and validated*	Pass
Emergency Requests	*Process the highest-priority first*	*Requests sorted by priority*	Pass
Route Navigation	*Find routes and shortest distance*	-	Pending
Audit History	*Display recent actions and save data*	*Latest actions displayed; data saved to file*	Pass

Results / Validation

Requests sorted by priority using bubble sort. Dijkstra's algorithm calculates the shortest route. Prevents invalid quantities and excess allocation. Stack displays recent actions first. Records saved to $disaster_data.txtusingfilehandling$.

GitHub Repository & Development Activity

Repository Information

- GitHub Link: *https://github.com/avikamittal2/Disaster-Management-System*
- README.md: *Updated*
- Contributors: *All Members Added*
- Repository Status: *Updated*
- Branches: *All code updates are currently managed in the main branch.*

Development Activity

- *Regular commits maintained*
- *Meaningful commit messages*
- *Individual contribution visible*
- *Latest implementation uploaded*
- *Documentation maintained*

Requirement: All team members must be contributors and must **commit individually using their own GitHub accounts**.

Individual Contribution

Member	Role	Technical Contribution	GitHub Commits
Avika	*System and Disaster Records Developer*	*Initial code structure, OOP design and Disaster Management module*	*7*
Aryan	*Resource Management Developer*	*Resource stock management and allocation with quantity validation*	*5*
Avni	*Emergency Response Developer*	*Emergency request management and priority-based sorting*	*5*
Gaurav	*Audit and Integration Developer*	*Audit history using a stack, file handling and main menu integration*	*4*

Challenges, Remaining Work & Phase-III Roadmap

Challenges Faced

- *Integrating modules*
- *Prioritizing emergency requests*
- *Implementing shortest-path routing*

Solutions / Learning

- *Sort requests by urgency using vectors*
- *Use modular classes and add file loading and saving for all modules*

Phase-III Roadmap

1. *Implement graph-based shortest paths*
2. *Improve file saving and loading*
3. *Test and debug all modules*
4. *Complete documentation and final demo*

Expected Outcomes & References

Expected Final Outcomes

- *Functional C++ disaster management system*
- *Efficient record and resource management*
- *Urgency-based request processing*
- *Improved route navigation*
- *Reliable data storage*

References

1. *NDMA Disaster Management Resources*
2. *Introduction to Algorithms (CLRS)*
3. *C++ Reference (cppreference)*
4. *GeeksforGeeks Data Structures Examples*

Thank You
Questions & Suggestions